

Scallop VeSCA Audit Report



contact@movebit.xyz



https://twitter.com/movebit_

Fri Mar 08 2024



Scallop VeSCA Audit Report

1 Executive Summary

1.1 Project Information

Description	Scallop VeSCA is a staking vesting protocol
Type	DeFi
Auditors	MoveBit
Timeline	Fri Mar 01 2024 - Fri Mar 08 2024
Languages	Move
Platform	Sui
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/scallop-io/ve-sca
Commits	63f14fdec4ecc294e273f15d902c0ee11223f170 63724ca5ed929f9f26f088db2b917c765e230431 91ad3dd79388ba139ee738f549916fbf4cb25344

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
MOV	Move.toml	3a74b79d7d6077b2a24e6b476f31b78ec9e34a76
UTI	sources/util.move	3feb5749e653ba2bff92f7bf8e8969adf719629c
MPR	Move.production.toml	e0168648fe6368b087b89cb2c9810fd830a398d9
VSUS	sources/ve_sca_unlock_schedule.move	a2d917131cdc1a7e53ffc06f6f33b2762a1fb15d
VSR	sources/ve_sca_rules.move	6a886402b473d1ea6d0ef735222f7761f4146435
CON	sources/constants.move	02f37ee21457f073accd609b35c62ffa6b20f239
CON1	sources/config.move	d9a17837fe97d0c6028975c0d8b9ef70a07ea38a
ECO	sources/error_code.move	67c3bd5a7bfea0e4310f3ce7a2feb0c78ccd963f
VST	sources/ve_sca_treasury.move	d2a767edcd080b652df565f651c0788ac30e6f6d
VSC	sources/ve_sca_calculator.move	ea6f1c2ebe7d795088086924d68164b6beea88a1
VSD	sources/ve_sca_display.move	ac2cb87c80dc3f4ec6949d1bcfe255d245b3587a

VSC1	sources/ve_sca.move	df2bbf2b38efbb08a963ec341bafc 87b3e52a41b
------	---------------------	--

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	5	5	0
Informational	0	0	0
Minor	2	2	0
Medium	0	0	0
Major	3	3	0
Critical	0	0	0

1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

1.5 Methodology

The security team adopted the "**Testing and Automated Analysis**", "**Code Review**" and "**Formal Verification**" strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Formal Verification

Perform formal verification for key functions with the Move Prover.

(4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by [Scallop VeSCA](#) to identify any potential issues and vulnerabilities in the source code of the [Scallop VeSCA](#) smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 5 issues of varying severity, listed below.

ID	Title	Severity	Status
CON-1	Optimizing Version Update Handling in the <code>upgrade_protocol_version()</code> Function	Minor	Fixed
ECO-1	The Error Codes Are Not Used Anywhere	Minor	Fixed
VSC-1	Oversight in <code>extend_lock_period()</code> Function May Lead to Anomalies	Major	Fixed
VST-1	The Lack of a Check for <code>ve_sca_amount = 0</code>	Major	Fixed
VST-2	Enhancing Contract Validation in <code>current_ve_sca_amount()</code>	Major	Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the [Scallop VeSCA](#) Smart Contract :

User

- User can mint VeSCA key, and lock the SCA into the treasury through `mint_ve_sca_key()`
- User can Lock more SCA for the VeSCA through `lock_more_sca()`
- User can extend the unlock time of the VeSCA through `extend_lock_period()`
- User can renew expired VeSCA through `renew_expired_ve_sca()`
- User can redeem the VeSCA for the locked SCA through `redeem()`

4 Findings

CON-1 Optimizing Version Update Handling in the `upgrade_protocol_version()` Function

Severity: Minor

Status: Fixed

Code Location:

`sources/config.move#53`

Descriptions:

In the function `upgrade_protocol_version()`, the method of incrementing the version number by 1 each time is used for version updates. However, if the value of `CURRENT_VE_SCA_PROTOCOL_VERSION` in the upgraded code contract is significantly higher than the previous version, for example, if the previous version is 5 and the code is updated with `CURRENT_VE_SCA_PROTOCOL_VERSION` changed to 10, calling `upgrade_protocol_version()` again after the contract upgrade will only increase the version by 1. The version check will fail, requiring multiple consecutive calls (in this case, 5) to achieve version consistency. This may not be an optimal approach. We recommend directly setting `config.version` to `CURRENT_VE_SCA_PROTOCOL_VERSION` in the `upgrade_protocol_version()` function and asserting that `config.version < CURRENT_VE_SCA_PROTOCOL_VERSION` before modification to ensure that the new version is greater than the old version. This approach may be more effective.

Suggestion:

It is recommended to set `config.version` directly to `CURRENT_VE_SCA_PROTOCOL_VERSION`.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

ECO-1 The Error Codes Are Not Used Anywhere

Severity: Minor

Status: Fixed

Code Location:

sources/error_code.move#4-5

Descriptions:

The error code functions here are not being utilized, resulting in additional gas consumption during deployment.

```
public fun cannot_topup_after_unlock(): u64 { 103 }  
public fun cannot_extend_after_unlock(): u64 { 104 }
```

Suggestion:

It is recommended to either utilize these error code functions or remove them to optimize gas usage during deployment.

Resolution:

This issue has been fixed. The client has already removed these error codes.

VSC-1 Oversight in `extend_lock_period()` Function May Lead to Anomalies

Severity: Major

Status: Fixed

Code Location:

`sources/ve_sca.move#140`

Descriptions:

The function `extend_lock_period()` is designed to extend the unlocking period. However, it lacks validation to ensure that the new time provided is greater than the previous unlocking time. If a smaller time value is passed, it may result in anomalies in subsequent calculations and withdrawals.

Suggestion:

It is recommended to add validation to ensure that `new_unlock_at` must be greater than the previous unlocking time.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

VST-1 The Lack of a Check for `ve_sca_amount = 0`

Severity: Major

Status: Fixed

Code Location:

`sources/ve_sca_treasury.move#54`

Descriptions:

The function `ve_sca_treasury.lock_sca()` is designed to lock the SCA tokens into the treasury and update the VeSCA amount. The `ve_sca_amount` is calculated as follows:

$$\text{ve_sca_amount} = \text{locked_sca_amount} * \text{remain_lock_period} / \text{max_lock_duration}$$

```
public fun calc_ve_sca(
  locked_sca_amount: u64,
  unlock_at: u64,
  clock: &Clock
): u64 {
  let now = clock::timestamp_ms(clock) / 1000;
  if (now >= unlock_at) {
    0
  } else {
    let remain_lock_period = unlock_at - now;
    mul_div(locked_sca_amount, remain_lock_period, constants::max_lock_duration())
  }
}
```

Here, `max_lock_duration` is 208 weeks, and `remain_lock_period` has a minimum value of 1 week. However, when the `locked_sca_amount` is less than 208, the calculated `ve_sca_amount` will be 0.

A bad actor could potentially exploit this by first calling `mint_ve_sca_key()` to obtain a `ve_sca_key` and then calling `lock_more_sca()` to lock more sca tokens each time, with each amount being less than 208. This way, the `ve_sca.locked_sca_amount` will be correctly updated,

```
let sca_amount = coin::value(&sca);  
ve_sca.locked_sca_amount = ve_sca.locked_sca_amount + sca_amount;
```

but the `treasury.current_ve_sca_amount` will increase by 0 each time, leading to incorrect accounting by the protocol.

```
// Increase the current VeSCA amount  
let ve_sca_amount = calculator::calc_ve_sca(sca_amount, unlock_at, clock);  
treasury.current_ve_sca_amount = treasury.current_ve_sca_amount + ve_sca_amount;
```

Suggestion:

It is recommended to add a check in the `calc_ve_sca()` function. If the calculated `ve_sca_amount` equals 0, revert directly.

Resolution:

This issue has been fixed. The code sets the minimum deposit amount for users and adds a check to ensure it is not zero.

VST-2 Enhancing Contract Validation in `current_ve_sca_amount()`

Severity: Major

Status: Fixed

Code Location:

`sources/ve_sca_treasury.move#137`

Descriptions:

The `current_ve_sca_amount()` function, being a public function, lacks validation for the contract version and status. This deficiency raises the risk of malicious accounts executing erroneous transactions, potentially leading to contract abnormalities.

Suggestion:

It is recommended to add validation for the contract version and status.

Resolution:

This issue has been fixed. The client has removed this function, use `total_ve_sca_amount()` instead.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.

